

Variable elimination on general graphs:

Generalizing chain inference

- Given, potentials $\psi_C(x_C)$, $C = \text{cliques in a graph } G$.
- Find, $Z = \sum_{x_1, \dots, x_n} \prod_C \psi_C(x_C)$

$x_1, \dots, x_n = \text{good ordering of variables}$

$\mathcal{F} = \{\psi_C(x_C) : C = \text{cliques in a graph } G\}$.

for $i = 1 \dots n$ **do**

$\mathcal{F}_i = \text{factors in } \mathcal{F} \text{ that contain } x_i$

$M_i = \text{product of factors in } \mathcal{F}_i$

$m_i = \sum_{x_i} M_i$

$\mathcal{F} = (\mathcal{F} - \mathcal{F}_i) \cup \{m_i\}$

end for

Example: Variable elimination

- Given, $\{\psi_{12}(x_1, x_2), \psi_{24}(x_2, x_4), \psi_{23}(x_2, x_3), \psi_{45}(x_4, x_5), \psi_{35}(x_3, x_5)\} \cup \{m_1(x_2)\} \cup \{m_2(x_3, x_4)\} \cup \{m_3(x_4, x_5)\}$

- Find, $Z =$

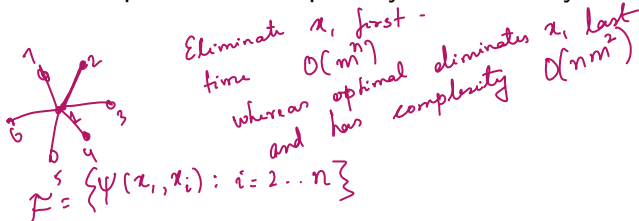
$$\sum_{x_1, \dots, x_5} \psi_{12}(x_1, x_2) \psi_{24}(x_2, x_4) \psi_{23}(x_2, x_3) \psi_{45}(x_4, x_5) \psi_{35}(x_3, x_5).$$

- $x_1: \prod\{\psi_{12}(x_1, x_2)\} \rightarrow M_1(x_1, x_2) \xrightarrow{\sum_{x_1}} m_1(x_2)$
- $x_2: \prod\{\psi_{24}(x_2, x_4), \psi_{23}(x_2, x_3), m_1(x_2)\} \rightarrow M_2(x_2, x_3, x_4) \xrightarrow{\sum_{x_2}} m_2(x_3, x_4)$
- $x_3: \prod\{\psi_{35}(x_3, x_5), m_2(x_3, x_4)\} \rightarrow M_3(x_3, x_4, x_5) \xrightarrow{\sum_{x_3}} m_3(x_4, x_5)$
- $x_4: \prod\{\psi_{45}(x_4, x_5), m_3(x_4, x_5)\} \rightarrow M_4(x_4, x_5) \xrightarrow{\sum_{x_4}} m_4(x_5)$
- $x_5: \prod\{m_4(x_5)\} \rightarrow M_5(x_5) \xrightarrow{\sum_{x_5}} Z$

Choosing a variable elimination order I

- Complexity of VE $O(nm^w)$ where w is the maximum number of variables in any factor.
- Wrong elimination order can give rise to very large intermediate factors.
- In the example above: eliminating x_2 first will give a factor of size 4. $O(nm^4)$

- Give an example where the penalty can be really severe



Choosing a variable elimination order II

- Choosing the optimal elimination order is NP hard for general graphs.
- Polynomial time algorithm exists for chordal graphs.
 - ▶ A graph is chordal or triangulated if all cycles of length greater than three have a shortcut.
- Optimal triangulation of graphs is NP hard. (Many heuristics)

Finding optimal order in a triangulated graph

Goal: find optimal ordering for $P(\underline{x_1})$ inference. $\underline{x_1}$ has to be last in the ordering.

Input: Graph G . n = number of vertices of G

for $i = 2, \dots, n$ **do**

π_i = pick any simplicial vertex in G other than 1.

remove π_i from G

end for

Return ordering $\pi_2, \pi_3, \dots, \pi_n$

Example

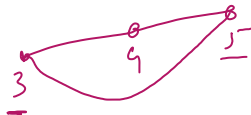
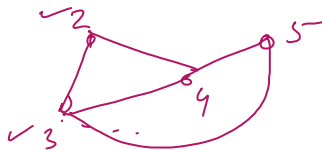
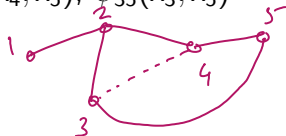
Given: $\psi_{12}(x_1, x_2)$, $\psi_{24}(x_2, x_4)$, $\psi_{23}(x_2, x_3)$, $\psi_{45}(x_4, x_5)$, $\psi_{35}(x_3, x_5)$
 compute $P(x_5)$

$$\pi_2 : 1$$

$$\pi_3 : 2$$

$$\pi_4 : 3$$

$$\pi_5 : 4$$



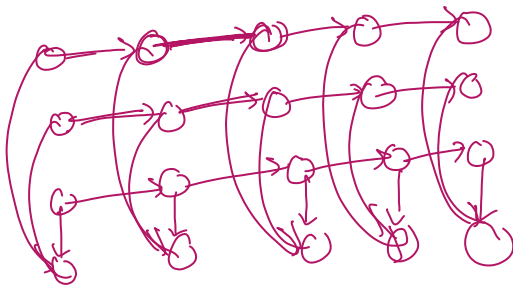
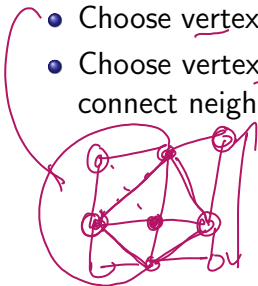
Triangulation heuristics

In the above algorithm, if no simplicial vertex found at a step, the graph is not triangulated.

Add extra edges to make the graph triangulated at that point.

The extra edges to be added can be chosen by one of these heuristics:

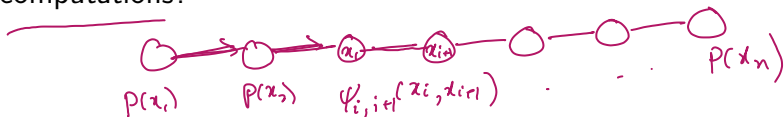
- Choose vertex with smallest degree and connect all its neighbors.
- Choose vertex which will require smallest additional edges to connect neighbors.



Reusing computation across multiple inference queries

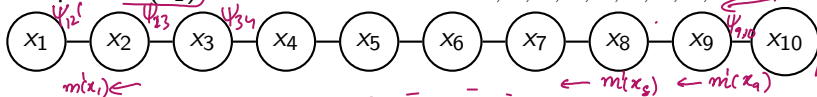
Given a chain graph with potentials $\psi_{i,i+1}(x_i, x_{i+1})$, suppose we need to compute all n marginals $P(x_1), \dots, P(x_n)$.

Invoking variable elimination algorithm n times for each x_i will entail a cost of $n \times nm^2$. Can we go faster by reusing work across computations?

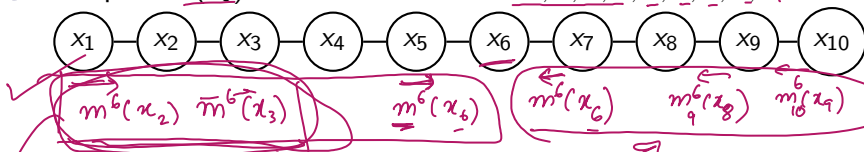


Example $n = 10$

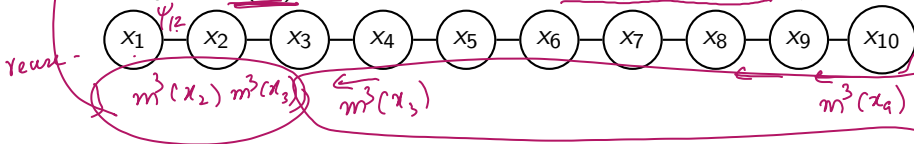
- 1 Compute $P(x_1)$: Eliminate in order 10, 9, 8, 7, 6, 5, 4, 3, 2



- 2 Compute $P(x_6)$: Eliminate in order 10, 9, 8, 7, 1, 2, 3, 4, 5.



- 3 Compute $P(x_3)$: Eliminate in order 10, 9, 8, 7, 6, 5, 4, 1, 2



Computation graph of VE algorithm

A special graph over cliques — Junction trees.

- 1 Find optimal ordering of variable elimination using the simplicial vertices ordering triangulating graph in the process if not.
- 2 Initial clique graph $\mathcal{C}_G$ is empty
- 3 Simulate running of VE algorithm. Each M_i is a node in $\mathcal{C}_G$. M_i is connected to all factors in $\mathcal{F}_i$ which were multiplied to form M_i .
- 4 Remove nodes that are subsumed by other nodes.
- 5 Assign each potential to any node in $\mathcal{C}_G$ that subsumes its variables.

Example: Junction tree from Variable elimination I

Given, $\{\psi_{12}(x_1, x_2), \psi_{24}(x_2, x_4), \psi_{23}(x_2, x_3), \psi_{45}(x_4, x_5), \psi_{35}(x_3, x_5)\}$

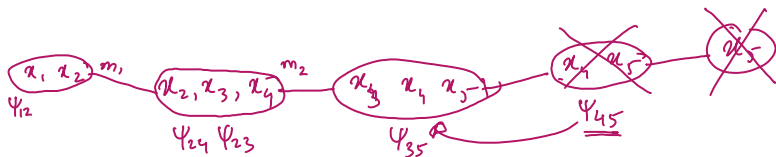
$$x_1: \prod\{\psi_{12}(x_1, x_2)\} \rightarrow M_1(x_1, x_2) \xrightarrow{\sum_{x_1}} m_1(x_2)$$

$$x_2: \prod\{\psi_{24}(x_2, x_4), \psi_{23}(x_2, x_3), m_1(x_2)\} \rightarrow M_2(x_2, x_3, x_4) \xrightarrow{\sum_{x_2}} m_2(x_3, x_4)$$

$$x_3: \prod\{\psi_{35}(x_3, x_5), m_2(x_3, x_4)\} \rightarrow M_3(x_3, x_4, x_5) \xrightarrow{\sum_{x_3}} m_3(x_4, x_5)$$

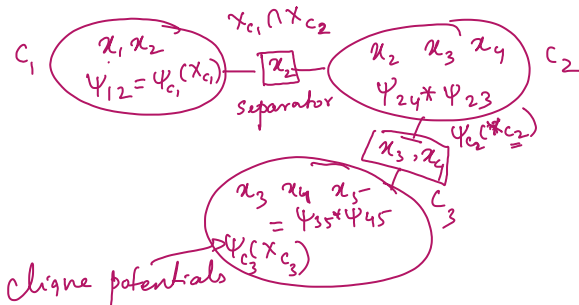
$$x_4: \prod\{\psi_{45}(x_4, x_5), m_3(x_4, x_5)\} \rightarrow M_4(x_4, x_5) \xrightarrow{\sum_{x_4}} m_4(x_5)$$

$$x_5: \prod\{m_4(x_5)\} \rightarrow M_5(x_5) \xrightarrow{\sum_{x_5}} Z$$



Example: Junction tree from Variable elimination II

JT:



Message passing algorithm

- An optimal general-purpose algorithm for exact marginal/MAP queries. Basis for several approximate inference algorithm too.
- Simultaneous computation of many queries
- Efficient data structures
- Complexity: $O(m^w N)$ $w =$ size of the largest clique in (triangulated) graph, $m =$ number of values of each discrete variable in the clique. → linear for trees.
- ~~Basis for many approximate algorithms.~~
- Many popular inference algorithms special cases of junction trees
 - ▶ Viterbi algorithm of HMMs
 - ▶ Forward-backward algorithm of Kalman filters

Message passing algorithm

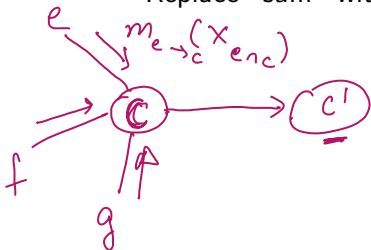
Given: Clique graph (CG) with cliques as nodes connected to other cliques.

- Until convergence repeat the following
 - ▶ Each node c sends message $m_{c \rightarrow c'}(.)$ to each of its neighbors c' once it has messages from every other neighbor $N(c) - \{c'\}$.

$$m_{c \rightarrow c'}(\mathbf{x}_s) = \sum_{\mathbf{x}_{c-s}} \psi_c(\mathbf{x}_c) \prod_{d \in N(c) - \{c'\}} m_{d \rightarrow c}(\mathbf{x}_{d \cap c})$$

$$s = c \cap c'$$

Replace “sum” with “max” for MAP queries.



Computing marginal probability of any variable x_i

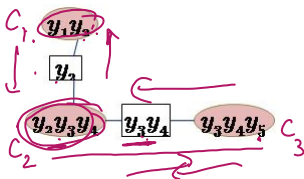
Marginal probability over all variables in a clique

$$\Pr(\mathbf{x}_c) \propto \psi_c(\mathbf{x}_c) \prod_{d \in N(c)} m_{d \rightarrow c}(\mathbf{x}_{d \cap c})$$

① c = clique in CG containing x_i

② $\Pr(x_i) = \sum_{\mathbf{x}_{c-x_i}} \Pr(\mathbf{x}_c)$

Example



$$\begin{aligned} \psi_{234}(\mathbf{y}_{234}) &= \psi_{23}(\mathbf{y}_{23})\psi_{34}(\mathbf{y}_{34}) \\ \psi_{345}(\mathbf{y}_{345}) &= \psi_{35}(\mathbf{y}_{35})\psi_{45}(\mathbf{y}_{45}) \\ \psi_{234}(\mathbf{y}_{12}) &= \psi_{12}(\mathbf{y}_{12}) \end{aligned}$$

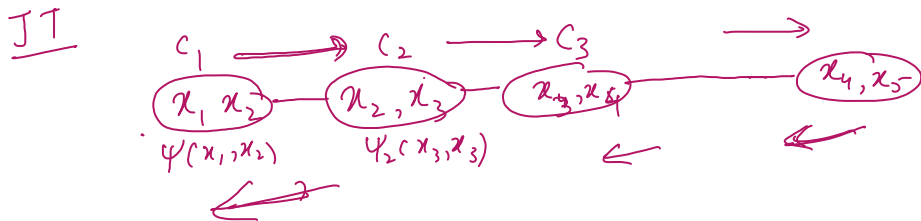
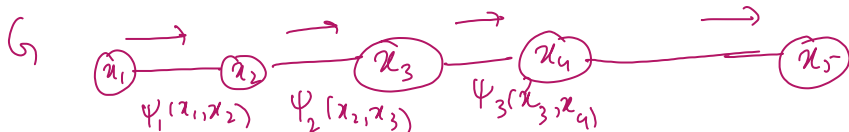
- 1 Clique "12" sends Message $m_{12 \rightarrow 234}(y_2) = \sum_{y_1} \psi_{12}(\mathbf{y}_{12})$ to its only neighbor.
- 2 Clique "345" sends Message $m_{345 \rightarrow 234}(\mathbf{y}_{34}) = \sum_{y_5} \psi_{345}(\mathbf{y}_{345})$ to "234"
- 3 Clique "234" sends Message $m_{234 \rightarrow 345}(\mathbf{y}_{34}) = \sum_{y_2} \psi_{234}(\mathbf{y}_{234}) m_{12 \rightarrow 234}(y_2)$ to "345"
- 4 Clique "234" sends Message $m_{234 \rightarrow 12}(y_2) = \sum_{y_4} \psi_{234}(\mathbf{y}_{234}) m_{345 \rightarrow 234}(\mathbf{y}_{34})$ to "12"

$$\Pr(y_1) \propto \sum_{y_2} \psi_{12}(\mathbf{y}_{12}) m_{234 \rightarrow 12}(y_2)$$

Special case: sum-product message passing over chain graphs (Baum Welch algorithm) I

Given potentials $\psi_i(x_i, x_{i+1})$ for $i = 1 \dots N - 1$

- Clique graph will have nodes as edges $(i, i + 1)$



Special case: sum-product message passing over chain graphs (Baum Welch algorithm) II

- For $i = 1 \dots N - 1$ (Forward pass)
 - Node $c = (i, i + 1)$ sends message $m_{c \rightarrow c'}(.)$ to right neighbor $c' = (i + 1, i + 2)$

$$m_{c \rightarrow c'}(\mathbf{x}_s) = \sum_{\mathbf{x}_{c-s}} \psi_c(\mathbf{x}_c) \prod_{d \in N(c) - \{c'\}} m_{d \rightarrow c}(\mathbf{x}_{d \cap c})$$

Becomes

$$\alpha_{i+1}(x_{i+1}) = \sum_{x_i} \psi(x_i, x_{i+1}) \alpha_i(x_i)$$

- For $i = N - 1 \dots 2$ (Backward pass)

Special case: sum-product message passing over chain graphs (Baum Welch algorithm) III

- Node $c = (i, i+1)$ sends message $m_{c \rightarrow c'}(.)$ to left neighbor $c' = (i-1, i)$

$$m_{c \rightarrow c'}(\mathbf{x}_s) = \sum_{\mathbf{x}_{c-s}} \psi_c(\mathbf{x}_c) \prod_{d \in N(c) - \{c'\}} m_{d \rightarrow c}(\mathbf{x}_{d \cap c})$$

Becomes

$$\beta_i(x_i) = \sum_{x_{i+1}} \psi(x_i, x_{i+1}) \beta_{i+1}(x_{i+1})$$

Computing marginal probability of any variable x_i

Marginal probability over all variables in a chain

$$\Pr(\mathbf{x}_c) \propto \psi_c(\mathbf{x}_c) \prod_{d \in N(c)} m_{d \rightarrow c}(\mathbf{x}_{d \cap c})$$

Becomes

$$\Pr(x_i, x_{i+1}) \propto \psi_i(x_i, x_{i+1}) \alpha_i(x_i) \beta_{i+1}(x_{i+1})$$

① $c = \text{clique in CG containing } x_i$

② $\Pr(x_i) = \sum_{\mathbf{x}_{c-x_i}} \Pr(\mathbf{x}_c)$

Becomes

① $c = \text{Either } (i, i+1) \text{ or } (i-1, i)$

Intuition behind message passing algorithm

Message from c to c' denotes the result of multiplying potentials on the side of the tree that contains clique c but not c' , and marginalizing all but the separator variables $s = c \cap c'$

